DATA STRUCTURES AND ALGORITHMS · 2026 ASSIGNMENT

Shortest Routes and Minimum Spanning Tree on the Blantyre City Road Network, from Limbe Market to Blantyre Market

A weighted graph of 22 places and 32 roads in Blantyre city, solved with Dijkstra's algorithm, Kruskal's algorithm and Prim's algorithm, implemented in C++ and deployed as a web application.

Group members

#	Name	Registration number
1	[Member 1 full name]	[Registration number]
2	[Member 2 full name]	[Registration number]
3	[Member 3 full name]	[Registration number]

Source code	https://github.com/mse23-mkapolo-cpu/DSA
Live demo	https://mse23-mkapolo-cpu.github.io/DSA/
Date	November 2026

Contents

1. The graph: Blantyre city road network (Q1)
2. Drawing and description of the graph (Q2)
3. How Dijkstra's algorithm works on the graph (Q3)
4. Implementation (Q4)
5. Testing and dry run (Q5)
6. Does the graph have an MST? (Q6)
7. Which MST algorithm and why (Q7)
8. Deployment (Q8)
9. Appendix: source code

1. The graph: Blantyre city road network

Q1 · 15 marks

Scenario. The graph models the roads of Blantyre city between **Limbe Market** in the east of the city and **Blantyre Market** in Blantyre town. Each **vertex** is a well-known place along these roads: markets, roundabouts, shopping malls, landmarks and institutions such as Limbe Health Centre, Independence Arch, Chichiri Shopping Mall, Red Cross Malawi and the Blantyre Clock Tower. Each **edge** is a road that links two places directly, without passing through another place in the graph. The **weight** of an edge is the road distance between the two places in kilometres.

The places and distances were chosen by the group and drawn by hand (section 2). The vertices are numbered 1 to 22 from Limbe Market (1) to Blantyre Market (22). There are three broad ways across the city: a northern route through Limbe Health Centre, St Columba and Stereo Night Club; a central route through Puma Energy Kanjedza, Automotive Products Limited and Kamba Pork Market; and a southern route through Yanakis Roundabout, Reunion Insurance Roundabout, Independence Arch and Chichiri Shopping Mall. Cross roads join the routes, so a traveller has many possible paths.

The graph is **undirected** because the roads carry traffic in both directions, so the distance from A to B equals the distance from B to A. It is **weighted** because the roads have different lengths, and it is **simple** (no self loops, at most one road between two places).

Distances are stored in the data file as whole metres (for example 0.45 km is stored as 450), and the program shows them in kilometres. Whole numbers avoid rounding errors when distances are added.

22	32	46.06	7.76
vertices (places)	edges (roads)	km of road in total	km, shortest route Limbe Market to Blantyre Market

Formally $G = (V, E, w)$ with $|V| = 22$, $|E| = 32$ and $w : E \to \mathbb{N}$ (metres). The full data is in `data/blantyre_roads.csv` in the repository, one road per line as `place_a,place_b,metres`.

Vertices and adjacency list

No.	Place	Area	Degree	Neighbours (distance in km)
1	Limbe Market	Limbe side	3	Limbe Health Centre (0.45), Liquid Lounge Roundabout (0.7), Yanakis Roundabout (1.2)
2	Limbe Health Centre	Limbe side	3	Limbe Market (0.45), Liquid Lounge Roundabout (0.35), St Columba (3.2)
3	Liquid Lounge Roundabout	Limbe side	4	Limbe Market (0.7), Limbe Health Centre (0.35), MASM (0.55), Puma Energy Kanjedza (1.0)
4	MASM	Limbe side	2	Liquid Lounge Roundabout (0.55), Yanakis Roundabout (1.0)
5	Yanakis Roundabout	Limbe side	3	Limbe Market (1.2), MASM (1.0), Reunion Insurance Roundabout (1.1)
6	Reunion Insurance Roundabout	Limbe side	2	Yanakis Roundabout (1.1), Independence Arch (1.0)
7	Independence Arch	Middle (Kanjedza–Chichiri)	3	Reunion Insurance Roundabout (1.0), Puma Energy Kanjedza (2.9), Chichiri Shopping Mall (2.9)

No.	Place	Area	Degree	Neighbours (distance in km)
8	Puma Energy Kanjedza	Limbe side	3	Liquid Lounge Roundabout (1.0), Automotive Products Limited (1.6), Independence Arch (2.9)
9	Automotive Products Limited	Middle (Kanjedza–Chichiri)	4	Puma Energy Kanjedza (1.6), St Columba (0.85), Kamba Pork Market (2.5), Chichiri Shopping Mall (1.5)
10	Chichiri Shopping Mall	Middle (Kanjedza–Chichiri)	4	Automotive Products Limited (1.5), Independence Arch (2.9), Red Cross Malawi (2.3), Castel (1.9)
11	St Columba	Middle (Kanjedza–Chichiri)	3	Limbe Health Centre (3.2), Automotive Products Limited (0.85), Stereo Night Club (3.1)
12	Kamba Pork Market	Middle (Kanjedza–Chichiri)	3	Automotive Products Limited (2.5), Stereo Night Club (0.3), Red Cross Malawi (1.2)
13	Stereo Night Club	Middle (Kanjedza–Chichiri)	3	St Columba (3.1), Kamba Pork Market (0.3), Rosewood Mall (0.7)
14	Rosewood Mall	Blantyre side	3	Stereo Night Club (0.7), Red Cross Malawi (1.4), CI Primary School (1.6)
15	Red Cross Malawi	Blantyre side	5	Kamba Pork Market (1.2), Rosewood Mall (1.4), Chichiri Shopping Mall (2.3), Mandala (0.55), Blantyre Clock Tower (3.4)
16	Castel	Blantyre side	2	Chichiri Shopping Mall (1.9), HHI (1.4)
17	HHI	Blantyre side	2	Castel (1.4), Blantyre Clock Tower (2.5)
18	Blantyre Clock Tower	Blantyre side	3	HHI (2.5), Red Cross Malawi (3.4), PA Nandos (1.2)
19	Mandala	Blantyre side	2	Red Cross Malawi (0.55), Blantyre Market (0.21)
20	PA Nandos	Blantyre side	2	Blantyre Clock Tower (1.2), Blantyre Market (1.2)
21	CI Primary School	Blantyre side	2	Rosewood Mall (1.6), Blantyre Market (0.3)
22	Blantyre Market	Blantyre side	3	CI Primary School (0.3), Mandala (0.21), PA Nandos (1.2)

Edges (weights in km)

Edge	Place A	Place B	km
e1	1 Limbe Market	2 Limbe Health Centre	0.45
e2	1 Limbe Market	3 Liquid Lounge Roundabout	0.7
e3	1 Limbe Market	5 Yanakis Roundabout	1.2
e4	2 Limbe Health Centre	3 Liquid Lounge Roundabout	0.35
e5	2 Limbe Health Centre	11 St Columba	3.2
e6	3 Liquid Lounge Roundabout	4 MASM	0.55

Edge	Place A	Place B	km
e17	11 St Columba	13 Stereo Night Club	3.1
e18	12 Kamba Pork Market	13 Stereo Night Club	0.3
e19	12 Kamba Pork Market	15 Red Cross Malawi	1.2
e20	13 Stereo Night Club	14 Rosewood Mall	0.7
e21	14 Rosewood Mall	15 Red Cross Malawi	1.4
e22	14 Rosewood Mall	21 CI Primary School	1.6

Edge	Place A	Place B	km
e7	3 Liquid Lounge Roundabout	8 Puma Energy Kanjedza	1.0
e8	4 MASM	5 Yanakis Roundabout	1.0
e9	5 Yanakis Roundabout	6 Reunion Insurance Roundabout	1.1
e10	6 Reunion Insurance Roundabout	7 Independence Arch	1.0
e11	8 Puma Energy Kanjedza	9 Automotive Products Limited	1.6
e12	8 Puma Energy Kanjedza	7 Independence Arch	2.9
e13	9 Automotive Products Limited	11 St Columba	0.85
e14	9 Automotive Products Limited	12 Kamba Pork Market	2.5
e15	9 Automotive Products Limited	10 Chichiri Shopping Mall	1.5
e16	7 Independence Arch	10 Chichiri Shopping Mall	2.9

Edge	Place A	Place B	km
e23	21 CI Primary School	22 Blantyre Market	0.3
e24	10 Chichiri Shopping Mall	15 Red Cross Malawi	2.3
e25	10 Chichiri Shopping Mall	16 Castel	1.9
e26	16 Castel	17 HHI	1.4
e27	17 HHI	18 Blantyre Clock Tower	2.5
e28	15 Red Cross Malawi	19 Mandala	0.55
e29	15 Red Cross Malawi	18 Blantyre Clock Tower	3.4
e30	19 Mandala	22 Blantyre Market	0.21
e31	18 Blantyre Clock Tower	20 PA Nandos	1.2
e32	20 PA Nandos	22 Blantyre Market	1.2

2. Drawing and description of the graph

Q2 · 10 marks

Paper drawing. Figure 1 is the group's hand-drawn graph. Each place is drawn as a numbered circle (vertex) with its name beside it. Each road is a straight line (edge) between two circles. Because the graph is undirected, the lines have no arrowheads. The distance in km is written along each line. Limbe Market (1) is on the left and Blantyre Market (22) on the right.

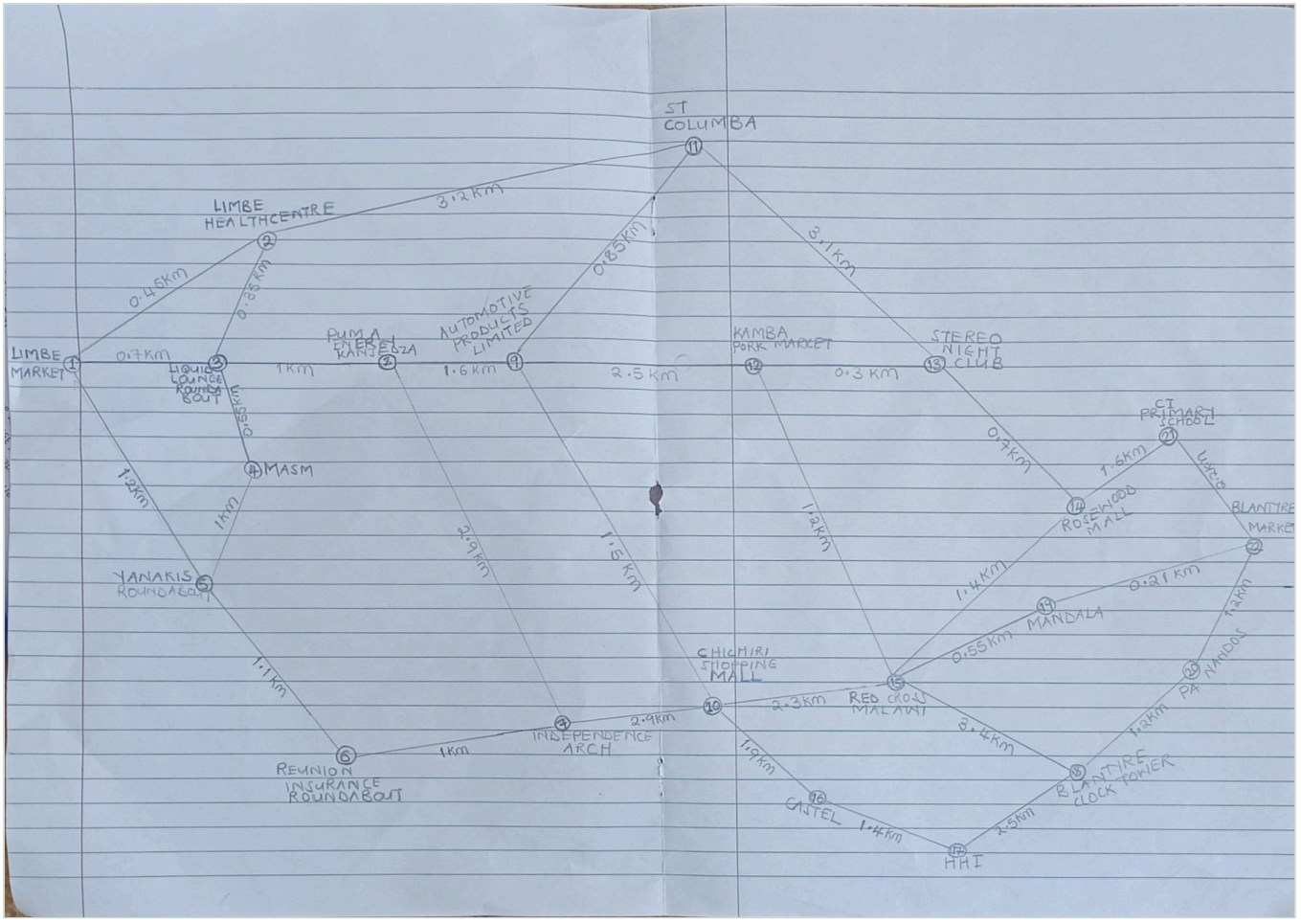


Figure 1. Hand-drawn graph, from Limbe Market (1) to Blantyre Market (22).

Figure 2 is the same graph drawn by the program, using the same layout. It is used for the rest of the report because it can highlight routes and trees.

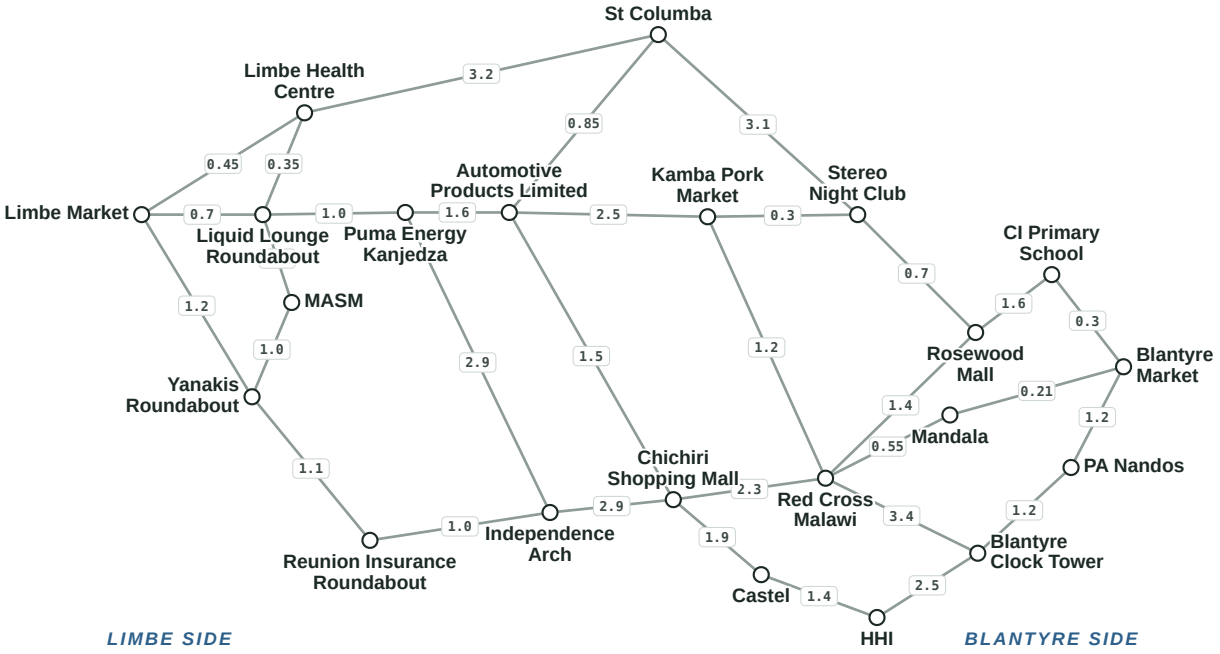


Figure 2. The graph drawn by the program, G = (V, E), |V| = 22, |E| = 32. Weights in km.

Description

- **Connected:** there is a path between every pair of places, so G has a single component. Test T10 confirms this: all 484 source–target pairs have a finite distance.
- **Sparse:** a simple graph on 22 vertices can have at most 22 × 21 / 2 = 231 edges. G has 32, a density of 32 / 231 ≈ 0.14. This is typical of road networks and is why an adjacency list is the right representation.
- **Degrees:** the sum of degrees is 2|E| = 64 (handshake lemma), so the average degree is 2.91. Red Cross Malawi has the highest degree (5): it is the hub where the central and southern routes meet before Blantyre. Liquid Lounge Roundabout, Automotive Products Limited and Chichiri Shopping Mall have degree 4. 7 places have degree 2: MASM, Reunion Insurance Roundabout, CI Primary School, Castel, HHI, Mandala, PA Nandos.
- **Cycles:** G contains many cycles, for example Limbe Market–Limbe Health Centre–Liquid Lounge Roundabout–Limbe Market (0.45 + 0.35 + 0.7 = 1.5 km) and Red Cross Malawi–Mandala–Blantyre Market–PA Nandos–Blantyre Clock Tower–Red Cross Malawi. The cyclomatic number |E| − |V| + 1 = 11 is the number of independent cycles, and also the number of roads that can be removed while keeping all places connected (see the MST in section 6).
- **No bridges:** removing any single road leaves the graph connected, because every road lies on a cycle. If one road is closed, for example for repairs, every place can still be reached by another route.
- **Longest and shortest roads:** Red Cross Malawi–Blantyre Clock Tower (3.4 km) is the longest. Mandala–Blantyre Market (0.21 km) is the shortest.

3. How Dijkstra's algorithm works on the graph

Q3 · 15 marks

Dijkstra's algorithm finds the shortest distance from one **source** vertex to every other vertex in a graph with non-negative weights. It keeps a tentative distance *d[v]* for every place, starting at 0 for the source and ∞ for the rest. At each step it **settles** the unsettled place with the smallest tentative distance; that distance is now final, because every other route would have to leave through a place that is already at least as far away and the weights are never negative. It then **relaxes** each road (u, v) out of the settled place: if d[u] + w(u, v) < d[v], it lowers d[v] and records u as the previous place on v's route.

```
DIJKSTRA(G, s)
  for each vertex v:  d[v] ← ∞,  prev[v] ← NIL
  d[s] ← 0;  Q ← min-priority queue containing (0, s)
  while Q is not empty:
      (du, u) ← EXTRACT-MIN(Q)
      if u is already settled: continue          // stale entry
      mark u settled
      for each edge (u, v, w):
          if v not settled and du + w < d[v]:
              d[v] ← du + w;  prev[v] ← u;  INSERT(Q, (d[v], v))
```

Worked example: all shortest routes from Limbe Market

We run the algorithm with Limbe Market as the source. The table shows, for each of the 22 steps, which place is settled, the roads that improved a tentative distance, and the reached-but-unsettled places afterwards (in order of distance, all in km). A place leaves the queue in exactly the order of its final distance.

Step	Settle	d (km)	Relaxations (old → new)	Tentative distances after the step
1	**Limbe Market**	0.0	Limbe Health Centre: ∞ → **0.45** Liquid Lounge Roundabout: ∞ → **0.7** Yanakis Roundabout: ∞ → **1.2**	Limbe Health Centre 0.45, Liquid Lounge Roundabout 0.7, Yanakis Roundabout 1.2
2	**Limbe Health Centre**	0.45	St Columba: ∞ → **3.65**	Liquid Lounge Roundabout 0.7, Yanakis Roundabout 1.2, St Columba 3.65
3	**Liquid Lounge Roundabout**	0.7	MASM: ∞ → **1.25** Puma Energy Kanjedza: ∞ → **1.7**	Yanakis Roundabout 1.2, MASM 1.25, Puma Energy Kanjedza 1.7, St Columba 3.65
4	**Yanakis Roundabout**	1.2	Reunion Insurance Roundabout: ∞ → **2.3**	MASM 1.25, Puma Energy Kanjedza 1.7, Reunion Insurance Roundabout 2.3, St Columba 3.65
5	**MASM**	1.25	none (no shorter road found)	Puma Energy Kanjedza 1.7, Reunion Insurance Roundabout 2.3, St Columba 3.65
6	**Puma Energy Kanjedza**	1.7	Automotive Products Limited: ∞ → **3.3** Independence Arch: ∞ → **4.6**	Reunion Insurance Roundabout 2.3, Automotive Products Limited 3.3, St Columba 3.65, Independence Arch 4.6
7	**Reunion Insurance Roundabout**	2.3	Independence Arch: 4.6 → **3.3**	Independence Arch 3.3, Automotive Products Limited 3.3, St Columba 3.65

Step	Settle	d (km)	Relaxations (old → new)	Tentative distances after the step
8	**Independence Arch**	3.3	Chichiri Shopping Mall: ∞ → **6.2**	Automotive Products Limited 3.3, St Columba 3.65, Chichiri Shopping Mall 6.2
9	**Automotive Products Limited**	3.3	Kamba Pork Market: ∞ → **5.8** Chichiri Shopping Mall: 6.2 → **4.8**	St Columba 3.65, Chichiri Shopping Mall 4.8, Kamba Pork Market 5.8
10	**St Columba**	3.65	Stereo Night Club: ∞ → **6.75**	Chichiri Shopping Mall 4.8, Kamba Pork Market 5.8, Stereo Night Club 6.75
11	**Chichiri Shopping Mall**	4.8	Red Cross Malawi: ∞ → **7.1** Castel: ∞ → **6.7**	Kamba Pork Market 5.8, Castel 6.7, Stereo Night Club 6.75, Red Cross Malawi 7.1
12	**Kamba Pork Market**	5.8	Stereo Night Club: 6.75 → **6.1** Red Cross Malawi: 7.1 → **7.0**	Stereo Night Club 6.1, Castel 6.7, Red Cross Malawi 7.0
13	**Stereo Night Club**	6.1	Rosewood Mall: ∞ → **6.8**	Castel 6.7, Rosewood Mall 6.8, Red Cross Malawi 7.0
14	**Castel**	6.7	HHI: ∞ → **8.1**	Rosewood Mall 6.8, Red Cross Malawi 7.0, HHI 8.1
15	**Rosewood Mall**	6.8	CI Primary School: ∞ → **8.4**	Red Cross Malawi 7.0, HHI 8.1, CI Primary School 8.4
16	**Red Cross Malawi**	7.0	Mandala: ∞ → **7.55** Blantyre Clock Tower: ∞ → **10.4**	Mandala 7.55, HHI 8.1, CI Primary School 8.4, Blantyre Clock Tower 10.4
17	**Mandala**	7.55	Blantyre Market: ∞ → **7.76**	Blantyre Market 7.76, HHI 8.1, CI Primary School 8.4, Blantyre Clock Tower 10.4
18	**Blantyre Market**	7.76	CI Primary School: 8.4 → **8.06** PA Nandos: ∞ → **8.96**	CI Primary School 8.06, HHI 8.1, PA Nandos 8.96, Blantyre Clock Tower 10.4
19	**CI Primary School**	8.06	none (no shorter road found)	HHI 8.1, PA Nandos 8.96, Blantyre Clock Tower 10.4
20	**HHI**	8.1	none (no shorter road found)	PA Nandos 8.96, Blantyre Clock Tower 10.4
21	**PA Nandos**	8.96	Blantyre Clock Tower: 10.4 → **10.16**	Blantyre Clock Tower 10.16
22	**Blantyre Clock Tower**	10.16	none (no shorter road found)	(empty)

Several steps show the algorithm correcting itself. In step 6, Puma Energy Kanjedza sets Independence Arch to 1.7 + 2.9 = 4.6 km. In step 7, Reunion Insurance Roundabout finds a shorter road, 2.3 + 1.0 = 3.3 km, so Independence Arch is updated to 3.3. In step 9, Automotive Products Limited lowers Chichiri Shopping Mall from 6.2 to 3.3 + 1.5 = 4.8 km. In step 12, Kamba Pork Market lowers Red Cross Malawi from 7.1 (via Chichiri) to 5.8 + 1.2 = 7.0 km. Steps such as MASM change nothing, because every neighbour already has a shorter distance.

Result: shortest distances and routes from Limbe Market

Place	Distance (km)	Previous place	Shortest route
Limbe Market	0.0	–	Limbe Market
Limbe Health Centre	0.45	Limbe Market	Limbe Market → Limbe Health Centre
Liquid Lounge Roundabout	0.7	Limbe Market	Limbe Market → Liquid Lounge Roundabout
Yanakis Roundabout	1.2	Limbe Market	Limbe Market → Yanakis Roundabout
MASM	1.25	Liquid Lounge Roundabout	Limbe Market → Liquid Lounge Roundabout → MASM
Puma Energy Kanjedza	1.7	Liquid Lounge Roundabout	Limbe Market → Liquid Lounge Roundabout → Puma Energy Kanjedza
Reunion Insurance Roundabout	2.3	Yanakis Roundabout	Limbe Market → Yanakis Roundabout → Reunion Insurance Roundabout
Independence Arch	3.3	Reunion Insurance Roundabout	Limbe Market → Yanakis Roundabout → Reunion Insurance Roundabout → Independence Arch
Automotive Products Limited	3.3	Puma Energy Kanjedza	Limbe Market → Liquid Lounge Roundabout → Puma Energy Kanjedza → Automotive Products Limited
St Columba	3.65	Limbe Health Centre	Limbe Market → Limbe Health Centre → St Columba
Chichiri Shopping Mall	4.8	Automotive Products Limited	Limbe Market → Liquid Lounge Roundabout → Puma Energy Kanjedza → Automotive Products Limited → Chichiri Shopping Mall
Kamba Pork Market	5.8	Automotive Products Limited	Limbe Market → Liquid Lounge Roundabout → Puma Energy Kanjedza → Automotive Products Limited → Kamba Pork Market
Stereo Night Club	6.1	Kamba Pork Market	Limbe Market → Liquid Lounge Roundabout → Puma Energy Kanjedza → Automotive Products Limited → Kamba Pork Market → Stereo Night Club
Castel	6.7	Chichiri Shopping Mall	Limbe Market → Liquid Lounge Roundabout → Puma Energy Kanjedza → Automotive Products Limited → Chichiri Shopping Mall → Castel
Rosewood Mall	6.8	Stereo Night Club	Limbe Market → Liquid Lounge Roundabout → Puma Energy Kanjedza → Automotive Products Limited → Kamba Pork Market → Stereo Night Club → Rosewood Mall
Red Cross Malawi	7.0	Kamba Pork Market	Limbe Market → Liquid Lounge Roundabout → Puma Energy Kanjedza → Automotive Products Limited → Kamba Pork Market → Red Cross Malawi
Mandala	7.55	Red Cross Malawi	Limbe Market → Liquid Lounge Roundabout → Puma Energy Kanjedza → Automotive Products Limited → Kamba Pork Market → Red Cross Malawi → Mandala
Blantyre Market	7.76	Mandala	Limbe Market → Liquid Lounge Roundabout → Puma Energy Kanjedza → Automotive Products Limited → Kamba Pork Market → Red Cross Malawi → Mandala → Blantyre Market

Place	Distance (km)	Previous place	Shortest route
CI Primary School	8.06	Blantyre Market	Limbe Market → Liquid Lounge Roundabout → Puma Energy Kanjedza → Automotive Products Limited → Kamba Pork Market → Red Cross Malawi → Mandala → Blantyre Market → CI Primary School
HHI	8.1	Castel	Limbe Market → Liquid Lounge Roundabout → Puma Energy Kanjedza → Automotive Products Limited → Chichiri Shopping Mall → Castel → HHI
PA Nandos	8.96	Blantyre Market	Limbe Market → Liquid Lounge Roundabout → Puma Energy Kanjedza → Automotive Products Limited → Kamba Pork Market → Red Cross Malawi → Mandala → Blantyre Market → PA Nandos
Blantyre Clock Tower	10.16	PA Nandos	Limbe Market → Liquid Lounge Roundabout → Puma Energy Kanjedza → Automotive Products Limited → Kamba Pork Market → Red Cross Malawi → Mandala → Blantyre Market → PA Nandos → Blantyre Clock Tower

The previous-place pointers form the **shortest-path tree** rooted at Limbe Market (Figure 3). Reading the route to any place means following the pointers back from that place to Limbe Market. For Blantyre Market: Blantyre Market ← Mandala ← Red Cross Malawi ← Kamba Pork Market ← Automotive Products Limited ← Puma Energy Kanjedza ← Liquid Lounge Roundabout ← Limbe Market, which gives

Limbe Market → Liquid Lounge Roundabout → Puma Energy Kanjedza → Automotive Products Limited → Kamba Pork Market → Red Cross Malawi → Mandala → Blantyre Market = 0.7 + 1.0 + 1.6 + 2.5 + 1.2 + 0.55 + 0.21 = 7.76 km

This is only 0.1 km shorter than going through Chichiri Shopping Mall (0.7 + 1.0 + 1.6 + 1.5 + 2.3 + 0.55 + 0.21 = 7.86 km). The southern route through Independence Arch and Chichiri is 3.3 + 2.9 + 2.3 + 0.55 + 0.21 = 9.26 km, and the northern route through Stereo Night Club, Rosewood Mall and CI Primary School is 3.3 + 2.5 + 0.3 + 0.7 + 1.6 + 0.3 = 8.7 km.

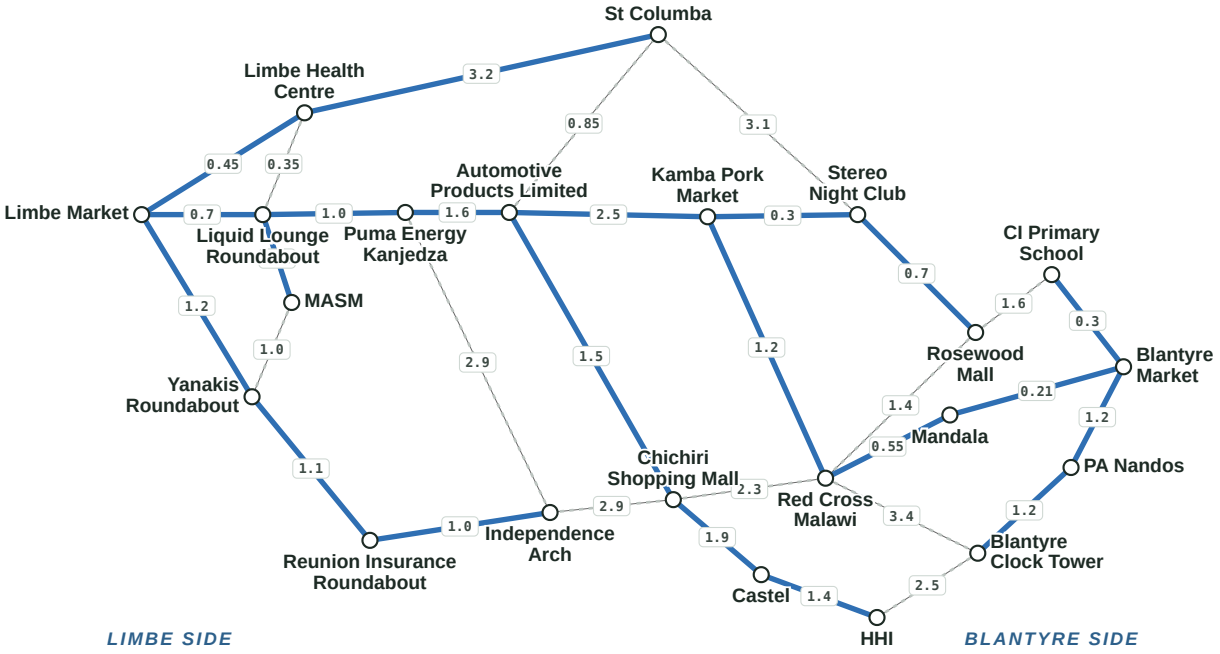


Figure 3. Shortest-path tree from Limbe Market (blue). Dashed roads are not used by any shortest route from Limbe Market.

4. Implementation

Q4 · 10 marks

The algorithm is implemented in **C++17** with only the standard library. The repository is organised as follows:

File	Purpose
`data/blantyre_roads.csv`	The graph: one road per line, `place_a,place_b,metres`.
`src/graph.hpp`	Graph class (adjacency list), Dijkstra, path reconstruction, Kruskal (with union–find) and Prim.
`src/main.cpp`	Command line program: shortest route, step-by-step trace, distances from a place, MST, and an interactive menu.
`tests/tests.cpp`	25 automated tests (section 5).
`docs/`	Web version of the same algorithms in JavaScript, published with GitHub Pages (section 8).
`Makefile`	`make` builds everything, `make test` runs the tests.

Design decisions

- **Adjacency list** (`vector<vector<pair<int, long long>>>`): uses O(V + E) memory and lets Dijkstra visit only the real neighbours of a place. An adjacency matrix would need 22 × 22 = 484 cells for 32 roads.
- **Place names are mapped to integer IDs** with a `std::map`, so the algorithms work on fast array indices while the user types names.
- **Distances in whole metres.** Storing 0.45 km as 450 m keeps all arithmetic in integers, so sums are exact and comparisons such as d[u] + w < d[v] never suffer from floating-point rounding. Output is converted to km.
- **Binary heap priority queue** (`std::priority_queue` with `std::greater`) holding (distance, place) pairs. Instead of a decrease-key operation, a place is pushed again when its distance improves, and outdated entries are skipped when popped (“lazy deletion”). This keeps the code short and gives **O((V + E) log V)** time.
- **Path reconstruction** uses the `prev[]` array, following pointers back from the target and reversing.
- **Input validation:** negative weights are rejected (Dijkstra is incorrect with them), as are self loops, malformed lines, non-numeric distances, missing files and unknown place names. Unreachable places are reported as “No route”.

Core code: Dijkstra

```
struct DijkstraResult {
    std::vector<long long> dist;  // dist[v] = shortest distance in metres from source (INF if
unreachable)
    std::vector<int> prev;        // prev[v] = previous place on the shortest path (-1 if none)
};

// One step of the algorithm, recorded for the dry run trace.
struct TraceStep {
    int settled;                                  // place removed from the queue and finalised
    long long settledDist;
    std::vector<std::tuple<int, long long, long long>> relaxed;  // (place, old dist, new dist)
};
```

```
// Dijkstra with a binary heap (std::priority_queue): O((V + E) log V).
inline DijkstraResult dijkstra(const Graph& g, int source, std::vector<TraceStep>* trace = nullptr) {
    int n = g.size();
    if (source < 0 || source >= n) throw std::out_of_range("Source index out of range");
    DijkstraResult r{std::vector<long long>(n, INF), std::vector<int>(n, -1)};
    std::vector<bool> done(n, false);
    using Item = std::pair<long long, int>;  // (distance, place)
    std::priority_queue<Item, std::vector<Item>, std::greater<Item>> pq;

    r.dist[source] = 0;
    pq.push({0, source});
    while (!pq.empty()) {
        auto [d, u] = pq.top();
        pq.pop();
        if (done[u]) continue;  // stale queue entry
        done[u] = true;
        TraceStep step{u, d, {}};
        for (auto [v, w] : g.neighbours(u)) {
            if (done[v]) continue;
            if (d + w < r.dist[v]) {
                step.relaxed.emplace_back(v, r.dist[v], d + w);
                r.dist[v] = d + w;
                r.prev[v] = u;
                pq.push({r.dist[v], v});
            }
        }
        if (trace) trace->push_back(step);
    }
    return r;
}

// Rebuild the path source -> target from prev[]. Empty if unreachable.
inline std::vector<int> buildPath(const DijkstraResult& r, int target) {
    std::vector<int> path;
    if (r.dist.at(target) >= INF) return path;
    for (int v = target; v != -1; v = r.prev[v]) path.push_back(v);
    std::reverse(path.begin(), path.end());
    return path;
}
```

Building and running

```
$ make                       # builds ./roads and ./run_tests
$ ./roads path "Limbe Market" "Blantyre Market"
Shortest route Limbe Market -> Blantyre Market: 7.76 km
  Limbe Market -> Liquid Lounge Roundabout -> Puma Energy Kanjedza -> Automotive Products Limited ->
Kamba Pork Market -> Red Cross Malawi -> Mandala -> Blantyre Market
$ ./roads trace "Red Cross Malawi" "Blantyre Clock Tower"   # step-by-step dry run
$ ./roads all "Limbe Market"        # distances from Limbe Market to every place
$ ./roads mst kruskal               # minimum spanning tree
$ ./roads                           # interactive menu
$ make test                         # run the 25 tests
```

5. Testing and dry run

Q5 · 20 marks

Test strategy

The tests in `tests/tests.cpp` cover four groups: (a) loading the graph, (b) known shortest routes worked out by hand, (c) an independent cross-check of every pair of places against a different algorithm (Floyd–Warshall), and (d) error handling and edge cases. Each test prints its input, the expected output and the actual output. The table below was produced by running `make test`.

Summary: 25 passed, 0 failed

ID	Test	Input	Expected	Actual	Result
T01	place and road count	data/blantyre_roads.csv	`22 places, 32 roads`	`22 places, 32 roads`	PASS
T02	same start and end	Mandala → Mandala	`0.0 km: Mandala`	`0.0 km: Mandala`	PASS
T03	direct neighbours	Mandala → Blantyre Market	`0.21 km: Mandala > Blantyre Market`	`0.21 km: Mandala > Blantyre Market`	PASS
T04	Limbe Market to Blantyre Market	Limbe Market → Blantyre Market	`7.76 km: Limbe Market > Liquid Lounge Roundabout > Puma Energy Kanjedza > Automotive Products Limited > Kamba Pork Market > Red Cross Malawi > Mandala > Blantyre Market`	`7.76 km: Limbe Market > Liquid Lounge Roundabout > Puma Energy Kanjedza > Automotive Products Limited > Kamba Pork Market > Red Cross Malawi > Mandala > Blantyre Market`	PASS
T05	Limbe Market to Chichiri	Limbe Market → Chichiri Shopping Mall	`4.8 km: Limbe Market > Liquid Lounge Roundabout > Puma Energy Kanjedza > Automotive Products Limited > Chichiri Shopping Mall`	`4.8 km: Limbe Market > Liquid Lounge Roundabout > Puma Energy Kanjedza > Automotive Products Limited > Chichiri Shopping Mall`	PASS
T06	southern route	Yanakis Roundabout → HHI	`8.3 km: Yanakis Roundabout > Reunion Insurance Roundabout > Independence Arch > Chichiri Shopping Mall > Castel > HHI`	`8.3 km: Yanakis Roundabout > Reunion Insurance Roundabout > Independence Arch > Chichiri Shopping Mall > Castel > HHI`	PASS
T07	symmetric (undirected)	Blantyre Market → Limbe Market	`7.76 km`	`7.76 km`	PASS
T08	northern start	St Columba → Mandala	`5.1 km: St Columba > Automotive Products Limited > Kamba Pork Market > Red Cross Malawi > Mandala`	`5.1 km: St Columba > Automotive Products Limited > Kamba Pork Market > Red Cross Malawi > Mandala`	PASS
T09	south to north-east	Reunion Insurance Roundabout → Stereo Night Club	`7.7 km: Reunion Insurance Roundabout > Independence Arch > Chichiri Shopping Mall > Red Cross Malawi > Kamba Pork Market > Stereo Night Club`	`7.7 km: Reunion Insurance Roundabout > Independence Arch > Chichiri Shopping Mall > Red Cross Malawi > Kamba Pork Market > Stereo Night Club`	PASS

ID	Test	Input	Expected	Actual	Result
T10	all pairs agree, all paths add up	every source and target	`0 mismatches`	`0 mismatches`	PASS
T11	unknown place	Limbe Market → Lilongwe	`error: Unknown place: Lilongwe`	`error: Unknown place: Lilongwe`	PASS
T12	place names are case sensitive	limbe market → Mandala	`error: Unknown place: limbe market`	`error: Unknown place: limbe market`	PASS
T13	negative distance rejected	addEdge(A, B, -500)	`error: Negative distance on road A - B: Dijkstra needs non-negative weights`	`error: Negative distance on road A - B: Dijkstra needs non-negative weights`	PASS
T14	distance not in whole metres	Mandala,Blantyre Market,0.21km	`error: Line 1: bad distance '0.21km'`	`error: Line 1: bad distance '0.21km'`	PASS
T15	wrong separator in file	Mandala;Blantyre Market;210	`error: Line 1: expected place_a,place_b,metres`	`error: Line 1: expected place_a,place_b,metres`	PASS
T16	missing data file	no_such_file.csv	`error: Cannot open no_such_file.csv`	`error: Cannot open no_such_file.csv`	PASS
T17	unreachable place	Limbe → Chileka	`unreachable`	`unreachable`	PASS
T18	MST of disconnected graph is a forest	{Limbe-Bangwe 2800, Chileka-Lunzu 9000}	`2 edges, total 11.8 km, spanning=false`	`2 edges, total 11.8 km, spanning=false`	PASS
T19	zero-length road allowed	A-B 0, B-C 400, A-C 900; A → C	`0.4 km: A > B > C`	`0.4 km: A > B > C`	PASS
T20	parallel roads: shorter one used	A-B 1000 and A-B 300; A → B	`0.3 km: A > B`	`0.3 km: A > B`	PASS
T21	Kruskal MST size	Blantyre graph	`21 edges, spanning=true`	`21 edges, spanning=true`	PASS
T22	Kruskal MST total	Blantyre graph	`20.66 km`	`20.66 km`	PASS
T23	Prim gives the same total	Blantyre graph, start at Limbe Market	`20.66 km`	`20.66 km`	PASS
T24	Prim from every start place	22 different start places	`0 mismatches`	`0 mismatches`	PASS
T25	MST connects all places	Kruskal edges	`1 component`	`1 component`	PASS

How the expected values were checked

Each expected route in T03–T09 was worked out by hand from the drawing and compared against the other possible routes. For T04 (Limbe Market → Blantyre Market) we compared the central route via Kamba Pork Market (7.76 km), the route via Chichiri Shopping Mall (7.86 km), the northern route via Stereo Night Club (8.7 km) and the southern route via Independence Arch (9.26 km). Test T10 then checks every one of the 484 pairs against Floyd–Warshall, a completely different algorithm, and checks that every route the program prints adds up to the distance it reports. This guards against mistakes in the hand calculations as well as in the code.

Errors

Tests T11–T16 check that bad input produces a clear error message instead of a crash or a wrong answer. The same errors appear in the command line program:

```
$ ./roads path "Limbe Market" Lilongwe
Error: Unknown place: Lilongwe
$ ./roads path "limbe market" Mandala
Error: Unknown place: limbe market
$ ./roads --data no_such_file.csv path "Limbe Market" Mandala
Error: Cannot open no_such_file.csv
```

Edge cases (T17–T20) use small separate graphs: a place with no road link to the rest (Chileka, joined only to Lunzu) is reported as unreachable, a zero-length road is allowed, and when two roads join the same places the shorter one is used. No test failed in the final run, and no crashes or incorrect results were found.

Dry run 1: Red Cross Malawi → Blantyre Clock Tower, tracing the priority queue

This dry run follows the code line by line, showing the real contents of the priority queue, including stale entries left behind by lazy deletion. Entries are written (place, distance in km) and the queue is shown sorted, smallest first.

Iter.	Action	Relaxing each road of the settled place	Priority queue after
init	–	dist[Red Cross Malawi] = 0, all others ∞	(Red Cross Malawi, 0.0)
1	pop (Red Cross Malawi, 0.0) settle Red Cross Malawi	Kamba Pork Market: 0.0 + 1.2 = 1.2 < ∞ → update, prev = Red Cross Malawi Rosewood Mall: 0.0 + 1.4 = 1.4 < ∞ → update, prev = Red Cross Malawi Chichiri Shopping Mall: 0.0 + 2.3 = 2.3 < ∞ → update, prev = Red Cross Malawi Mandala: 0.0 + 0.55 = 0.55 < ∞ → update, prev = Red Cross Malawi Blantyre Clock Tower: 0.0 + 3.4 = 3.4 < ∞ → update, prev = Red Cross Malawi	(Mandala, 0.55) (Kamba Pork Market, 1.2) (Rosewood Mall, 1.4) (Chichiri Shopping Mall, 2.3) (Blantyre Clock Tower, 3.4)
2	pop (Mandala, 0.55) settle Mandala	Red Cross Malawi: settled, skip Blantyre Market: 0.55 + 0.21 = 0.76 < ∞ → update, prev = Mandala	(Blantyre Market, 0.76) (Kamba Pork Market, 1.2) (Rosewood Mall, 1.4) (Chichiri Shopping Mall, 2.3) (Blantyre Clock Tower, 3.4)

Iter.	Action	Relaxing each road of the settled place	Priority queue after
3	pop (Blantyre Market, 0.76) settle Blantyre Market	CI Primary School: 0.76 + 0.3 = 1.06 < ∞ → update, prev = Blantyre Market Mandala: settled, skip PA Nandos: 0.76 + 1.2 = 1.96 < ∞ → update, prev = Blantyre Market	(CI Primary School, 1.06) (Kamba Pork Market, 1.2) (Rosewood Mall, 1.4) (PA Nandos, 1.96) (Chichiri Shopping Mall, 2.3) (Blantyre Clock Tower, 3.4)
4	pop (CI Primary School, 1.06) settle CI Primary School	Rosewood Mall: 1.06 + 1.6 = 2.66 ≥ 1.4 → no change Blantyre Market: settled, skip	(Kamba Pork Market, 1.2) (Rosewood Mall, 1.4) (PA Nandos, 1.96) (Chichiri Shopping Mall, 2.3) (Blantyre Clock Tower, 3.4)
5	pop (Kamba Pork Market, 1.2) settle Kamba Pork Market	Automotive Products Limited: 1.2 + 2.5 = 3.7 < ∞ → update, prev = Kamba Pork Market Stereo Night Club: 1.2 + 0.3 = 1.5 < ∞ → update, prev = Kamba Pork Market Red Cross Malawi: settled, skip	(Rosewood Mall, 1.4) (Stereo Night Club, 1.5) (PA Nandos, 1.96) (Chichiri Shopping Mall, 2.3) (Blantyre Clock Tower, 3.4) (Automotive Products Limited, 3.7)
6	pop (Rosewood Mall, 1.4) settle Rosewood Mall	Stereo Night Club: 1.4 + 0.7 = 2.1 ≥ 1.5 → no change Red Cross Malawi: settled, skip CI Primary School: settled, skip	(Stereo Night Club, 1.5) (PA Nandos, 1.96) (Chichiri Shopping Mall, 2.3) (Blantyre Clock Tower, 3.4) (Automotive Products Limited, 3.7)
7	pop (Stereo Night Club, 1.5) settle Stereo Night Club	St Columba: 1.5 + 3.1 = 4.6 < ∞ → update, prev = Stereo Night Club Kamba Pork Market: settled, skip Rosewood Mall: settled, skip	(PA Nandos, 1.96) (Chichiri Shopping Mall, 2.3) (Blantyre Clock Tower, 3.4) (Automotive Products Limited, 3.7) (St Columba, 4.6)
8	pop (PA Nandos, 1.96) settle PA Nandos	Blantyre Clock Tower: 1.96 + 1.2 = 3.16 < 3.4 → update, prev = PA Nandos Blantyre Market: settled, skip	(Chichiri Shopping Mall, 2.3) (Blantyre Clock Tower, 3.16) (Blantyre Clock Tower, 3.4) (Automotive Products Limited, 3.7) (St Columba, 4.6)
9	pop (Chichiri Shopping Mall, 2.3) settle Chichiri Shopping Mall	Automotive Products Limited: 2.3 + 1.5 = 3.8 ≥ 3.7 → no change Independence Arch: 2.3 + 2.9 = 5.2 < ∞ → update, prev = Chichiri Shopping Mall Red Cross Malawi: settled, skip Castel: 2.3 + 1.9 = 4.2 < ∞ → update, prev = Chichiri Shopping Mall	(Blantyre Clock Tower, 3.16) (Blantyre Clock Tower, 3.4) (Automotive Products Limited, 3.7) (Castel, 4.2) (St Columba, 4.6) (Independence Arch, 5.2)
10	pop (Blantyre Clock Tower, 3.16) settle Blantyre Clock Tower	HHI: 3.16 + 2.5 = 5.66 < ∞ → update, prev = Blantyre Clock Tower Red Cross Malawi: settled, skip PA Nandos: settled, skip **Target settled: stop.**	(Blantyre Clock Tower, 3.4) (Automotive Products Limited, 3.7) (Castel, 4.2) (St Columba, 4.6) (Independence Arch, 5.2) (HHI, 5.66)

This dry run is interesting because the direct road from Red Cross Malawi to Blantyre Clock Tower (3.4 km) is **not** the shortest route. The clock tower first gets distance 3.4 in iteration 1, then PA Nandos improves it to 3.16 in iteration 8. The old entry (Blantyre Clock Tower, 3.4) stays in the queue as a stale entry, but the target is settled at 3.16 before it is ever popped.

Result: following prev[] back from Blantyre Clock Tower gives Red Cross Malawi → Mandala → Blantyre Market → PA Nandos → Blantyre Clock Tower, a distance of **3.16 km**. The program prints the same:

```
$ ./roads path "Red Cross Malawi" "Blantyre Clock Tower"
Shortest route Red Cross Malawi -> Blantyre Clock Tower: 3.16 km
  Red Cross Malawi -> Mandala -> Blantyre Market -> PA Nandos -> Blantyre Clock Tower
```

Dry run 2: program trace for Limbe Market → Blantyre Market

The `trace` command prints each step the algorithm takes and stops once the destination is settled.

```
$ ./roads trace "Limbe Market" "Blantyre Market"
Dry run of Dijkstra from Limbe Market
Step  1: settle Limbe Market (0.0 km)
          update Limbe Health Centre: inf -> 0.45
          update Liquid Lounge Roundabout: inf -> 0.7
          update Yanakis Roundabout: inf -> 1.2
Step  2: settle Limbe Health Centre (0.45 km)
          update St Columba: inf -> 3.65
Step  3: settle Liquid Lounge Roundabout (0.7 km)
          update MASM: inf -> 1.25
          update Puma Energy Kanjedza: inf -> 1.7
Step  4: settle Yanakis Roundabout (1.2 km)
          update Reunion Insurance Roundabout: inf -> 2.3
Step  5: settle MASM (1.25 km), no updates
Step  6: settle Puma Energy Kanjedza (1.7 km)
          update Automotive Products Limited: inf -> 3.3
          update Independence Arch: inf -> 4.6
Step  7: settle Reunion Insurance Roundabout (2.3 km)
          update Independence Arch: 4.6 -> 3.3
Step  8: settle Independence Arch (3.3 km)
          update Chichiri Shopping Mall: inf -> 6.2
Step  9: settle Automotive Products Limited (3.3 km)
          update Kamba Pork Market: inf -> 5.8
          update Chichiri Shopping Mall: 6.2 -> 4.8
Step 10: settle St Columba (3.65 km)
          update Stereo Night Club: inf -> 6.75
Step 11: settle Chichiri Shopping Mall (4.8 km)
          update Red Cross Malawi: inf -> 7.1
          update Castel: inf -> 6.7
Step 12: settle Kamba Pork Market (5.8 km)
          update Stereo Night Club: 6.75 -> 6.1
          update Red Cross Malawi: 7.1 -> 7.0
Step 13: settle Stereo Night Club (6.1 km)
          update Rosewood Mall: inf -> 6.8
Step 14: settle Castel (6.7 km)
          update HHI: inf -> 8.1
Step 15: settle Rosewood Mall (6.8 km)
          update CI Primary School: inf -> 8.4
Step 16: settle Red Cross Malawi (7.0 km)
          update Mandala: inf -> 7.55
          update Blantyre Clock Tower: inf -> 10.4
Step 17: settle Mandala (7.55 km)
          update Blantyre Market: inf -> 7.76
Step 18: settle Blantyre Market (7.76 km)
          update CI Primary School: 8.4 -> 8.06
          update PA Nandos: inf -> 8.96
Target Blantyre Market settled, so its distance is final.

Shortest route Limbe Market -> Blantyre Market: 7.76 km
  Limbe Market -> Liquid Lounge Roundabout -> Puma Energy Kanjedza -> Automotive Products Limited ->
Kamba Pork Market -> Red Cross Malawi -> Mandala -> Blantyre Market
```

6. Does the graph have a minimum spanning tree?

Q6 · 10 marks

Yes. A spanning tree is a subset of the edges that connects all the vertices with no cycles; it has exactly |V| − 1 edges. A minimum spanning tree (MST) is a spanning tree with the smallest possible total weight. An undirected graph has a spanning tree if and only if it is **connected**, and then it also has a minimum one, because there are finitely many spanning trees and one of them must have the smallest total. Our graph is undirected, weighted and connected (section 2), so it has an MST.

The MST of our graph has **21 edges** and a total length of **20.66 km**, compared with 46.06 km for all 32 roads. The other 11 roads (the cyclomatic number from section 2) each close a cycle and are not needed for connectivity.

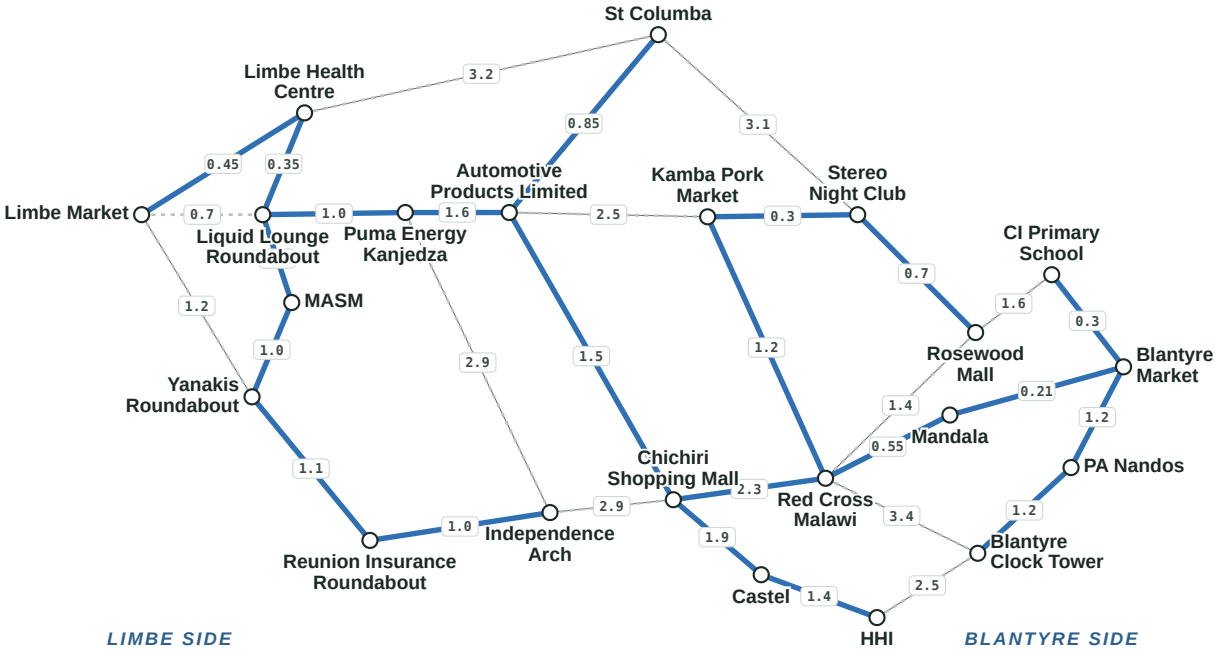


Figure 4. Minimum spanning tree (blue, 20.66 km). Dashed roads are not in the MST.

The MST is unique. Some roads have equal lengths (for example several roads of 1.0 km and of 1.2 km), which can sometimes allow more than one MST. We checked every road that is not in the tree: in each case it is strictly longer than every road on the tree path between its two places, so no swap gives an equal total. The MST is therefore the only one.

What the MST represents

The MST is the **shortest total length of road that still connects every place** to every other place. For example, it does not use the direct road Limbe Market–Liquid Lounge Roundabout (0.7 km), because the two places are already joined through Limbe Health Centre with shorter roads (0.45 + 0.35 km). Between Limbe Market and Blantyre Market this answers questions such as:

- **Laying pipes or cables along the roads.** Blantyre Water Board pipes, ESCOM power lines or a fibre-optic network are usually laid along road reserves. Connecting all 22 places along the MST needs 20.66 km of pipe or cable instead of 46.06 km.
- **Road maintenance on a limited budget.** If Blantyre City Council could only maintain 21 roads, the MST keeps every place from Limbe Market to Blantyre Market reachable for the least total length of road.

- **Emergency access.** During heavy rains, when some roads flood, the MST is a minimal backbone that must stay open so that every place, including Limbe Health Centre and Red Cross Malawi, can still be reached.

What the MST does not represent: it is not a set of shortest routes. In the MST the only route from Limbe Market to Blantyre Market is Limbe Market → Limbe Health Centre → Liquid Lounge Roundabout → Puma Energy Kanjedza → Automotive Products Limited → Chichiri Shopping Mall → Red Cross Malawi → Mandala → Blantyre Market, which is **7.96 km**, compared with the 7.76 km found by Dijkstra. The MST minimises the total length of the whole network; Dijkstra minimises the length of one journey.

7. Which MST algorithm we would use, and why

Q7 · 10 marks

	Kruskal	Prim
Idea	Sort all roads by length. Add each road unless it joins two places that are already connected (it would make a cycle).	Start from one place. Repeatedly add the shortest road from the tree to a place not yet in the tree.
Data structure	Sorted edge list + union–find (disjoint sets)	Priority queue + adjacency list
Time	O(E log E)	O(E log V) with a binary heap; O(V^2) with an array
Best for	Sparse graphs, data given as an edge list	Dense graphs, data given as an adjacency matrix
On our graph	32 × $\log_2 32$ ≈ 160 basic steps	32 × $\log_2 22$ ≈ 143 basic steps

We would use Kruskal's algorithm, for these reasons:

1. **The graph is sparse** (32 edges, density 0.14). Kruskal's cost depends on sorting the edges, which is cheap when there are few of them. The O(V^2) array version of Prim, which suits dense graphs, would waste work here. For a graph this small the binary-heap versions of both algorithms are practically equal in speed, so the decision comes down to the next points.
2. **Our data is already an edge list.** The file `blantyre_roads.csv` is a list of roads with lengths, which is exactly Kruskal's input. No adjacency structure is needed.
3. **It is easy to explain and check by hand.** Kruskal processes roads from shortest to longest and makes one yes/no decision for each, as in the trace below. This matches how a planner would think: "lay the cheapest links first, unless the two places are already connected".
4. **It handles a disconnected network gracefully.** If floods cut the network into pieces (test T18), Kruskal returns a minimum spanning forest, one tree per piece, without any change. Prim only builds the tree of the piece containing its start place.

We implemented **both** algorithms to confirm the answer. Prim gives the same 20.66 km total from every one of the 22 possible start places (test T24).

Kruskal trace on our graph

#	Road (sorted by length)	km	Decision
1	Mandala – Blantyre Market	0.21	add
2	Kamba Pork Market – Stereo Night Club	0.3	add
3	CI Primary School – Blantyre Market	0.3	add
4	Limbe Health Centre – Liquid Lounge Roundabout	0.35	add
5	Limbe Market – Limbe Health Centre	0.45	add
6	Liquid Lounge Roundabout – MASM	0.55	add
7	Red Cross Malawi – Mandala	0.55	add
8	Limbe Market – Liquid Lounge Roundabout	0.7	skip: Limbe Market and Liquid Lounge Roundabout already connected (cycle)
9	Stereo Night Club – Rosewood Mall	0.7	add

#	Road (sorted by length)	km	Decision
10	Automotive Products Limited – St Columba	0.85	add
11	Liquid Lounge Roundabout – Puma Energy Kanjedza	1.0	add
12	MASM – Yanakis Roundabout	1.0	add
13	Reunion Insurance Roundabout – Independence Arch	1.0	add
14	Yanakis Roundabout – Reunion Insurance Roundabout	1.1	add
15	Limbe Market – Yanakis Roundabout	1.2	skip: Limbe Market and Yanakis Roundabout already connected (cycle)
16	Kamba Pork Market – Red Cross Malawi	1.2	add
17	Blantyre Clock Tower – PA Nandos	1.2	add
18	PA Nandos – Blantyre Market	1.2	add
19	Rosewood Mall – Red Cross Malawi	1.4	skip: Rosewood Mall and Red Cross Malawi already connected (cycle)
20	Castel – HHI	1.4	add
21	Automotive Products Limited – Chichiri Shopping Mall	1.5	add
22	Puma Energy Kanjedza – Automotive Products Limited	1.6	add
23	Rosewood Mall – CI Primary School	1.6	skip: Rosewood Mall and CI Primary School already connected (cycle)
24	Chichiri Shopping Mall – Castel	1.9	add
25	Chichiri Shopping Mall – Red Cross Malawi	2.3	add
26	Automotive Products Limited – Kamba Pork Market	2.5	skip: Automotive Products Limited and Kamba Pork Market already connected (cycle)
27	HHI – Blantyre Clock Tower	2.5	skip: HHI and Blantyre Clock Tower already connected (cycle)
28	Puma Energy Kanjedza – Independence Arch	2.9	skip: Puma Energy Kanjedza and Independence Arch already connected (cycle)
29	Independence Arch – Chichiri Shopping Mall	2.9	skip: Independence Arch and Chichiri Shopping Mall already connected (cycle)
30	St Columba – Stereo Night Club	3.1	skip: St Columba and Stereo Night Club already connected (cycle)
31	Limbe Health Centre – St Columba	3.2	skip: Limbe Health Centre and St Columba already connected (cycle)
32	Red Cross Malawi – Blantyre Clock Tower	3.4	skip: Red Cross Malawi and Blantyre Clock Tower already connected (cycle)

The tree is complete after 21 additions (the last is Chichiri Shopping Mall–Red Cross Malawi, 2.3 km). The algorithm still scans the remaining longer roads but none can be added. Total: **20.66 km**.

Prim's trace from Limbe Market (for comparison)

#	Shortest road leaving the tree	km
1	Limbe Market → Limbe Health Centre	0.45
2	Limbe Health Centre → Liquid Lounge Roundabout	0.35

#	Shortest road leaving the tree	km
3	Liquid Lounge Roundabout → MASM	0.55
4	Liquid Lounge Roundabout → Puma Energy Kanjedza	1.0
5	MASM → Yanakis Roundabout	1.0
6	Yanakis Roundabout → Reunion Insurance Roundabout	1.1
7	Reunion Insurance Roundabout → Independence Arch	1.0
8	Puma Energy Kanjedza → Automotive Products Limited	1.6
9	Automotive Products Limited → St Columba	0.85
10	Automotive Products Limited → Chichiri Shopping Mall	1.5
11	Chichiri Shopping Mall → Castel	1.9
12	Castel → HHI	1.4
13	Chichiri Shopping Mall → Red Cross Malawi	2.3
14	Red Cross Malawi → Mandala	0.55
15	Mandala → Blantyre Market	0.21
16	Blantyre Market → CI Primary School	0.3
17	Red Cross Malawi → Kamba Pork Market	1.2
18	Kamba Pork Market → Stereo Night Club	0.3
19	Stereo Night Club → Rosewood Mall	0.7
20	Blantyre Market → PA Nandos	1.2
21	PA Nandos → Blantyre Clock Tower	1.2

Total: **20.66 km**, the same tree as Kruskal, built in a different order.

8. Deployment

Q8 · 10 marks

The work is deployed in two ways, so it can be tested with nothing to install.

Web application (no installation)

The same Dijkstra, Kruskal and Prim algorithms are written in JavaScript in `docs/graph.js` and run in the browser. The page is hosted free on GitHub Pages:

https://mse23-mkapolo-cpu.github.io/DSA/

- **Shortest route:** choose any two places. The page shows the distance, the route highlighted on the map, and the full step-by-step table of the algorithm with the queue after each step.
- **Spanning tree:** shows the MST on the map with the Kruskal decision table, or Prim's order.
- **Tests:** runs the same test cases as the C++ suite in the browser and shows pass or fail for each.

C++ program

The C++ source is on GitHub at https://github.com/mse23-mkapolo-cpu/DSA. It can be run without installing anything by opening the repository in GitHub Codespaces (Code → Codespaces → Create), then running `make && ./roads` in the terminal. Alternatively, `src/graph.hpp` and `src/main.cpp` can be pasted into an online C++ compiler such as OnlineGDB. On a machine with g++ the steps are:

```
git clone https://github.com/mse23-mkapolo-cpu/DSA.git
cd DSA
make && make test
./roads path "Limbe Market" "Blantyre Market"
```

Appendix A: src/graph.hpp

```
// graph.hpp - Weighted undirected graph of places in Blantyre city, with
// Dijkstra's shortest path and two Minimum Spanning Tree algorithms
// (Prim and Kruskal).
#pragma once

#include <algorithm>
#include <fstream>
#include <functional>
#include <limits>
#include <map>
#include <queue>
#include <sstream>
#include <stdexcept>
#include <string>
#include <tuple>
#include <vector>

namespace mw {

const long long INF = std::numeric_limits<long long>::max() / 4;

struct Edge {
    int u, v;
    long long w;
};

class Graph {
public:
    // Add a place if it is new; return its index either way.
    int addNode(const std::string& name) {
        auto it = index_.find(name);
        if (it != index_.end()) return it->second;
        int id = static_cast<int>(names_.size());
        names_.push_back(name);
        index_[name] = id;
        adj_.emplace_back();
        return id;
    }

    void addEdge(const std::string& a, const std::string& b, long long w) {
        if (w < 0) throw std::invalid_argument("Negative distance on road " + a + " - " + b +
                                               ": Dijkstra needs non-negative weights");
        if (a == b) throw std::invalid_argument("Self loop on " + a + " is not allowed");
        int u = addNode(a), v = addNode(b);
        adj_[u].push_back({v, w});
        adj_[v].push_back({u, w});
        edges_.push_back({u, v, w});
    }

    // Load "place_a,place_b,metres" lines. Blank lines and lines starting with # are skipped.
    static Graph fromCsv(std::istream& in) {
        Graph g;
        std::string line;
        int lineNo = 0;
        while (std::getline(in, line)) {
            ++lineNo;
            if (!line.empty() && line.back() == '\r') line.pop_back();
            if (line.empty() || line[0] == '#') continue;
            std::stringstream ss(line);
            std::string a, b, w;
            if (!std::getline(ss, a, ',') || !std::getline(ss, b, ',') || !std::getline(ss, w)) {
                throw std::runtime_error("Line " + std::to_string(lineNo) + ": expected place_a,place_b,metres");
            }
            long long metres;
            try {
                size_t used = 0;
                metres = std::stoll(w, &used);
                if (used != w.size()) throw std::invalid_argument("trailing");
            } catch (const std::exception&) {
                throw std::runtime_error("Line " + std::to_string(lineNo) + ": bad distance '" + w + "'");
            }
            g.addEdge(a, b, metres);
        }
        return g;
```

```
    }

    static Graph fromFile(const std::string& path) {
        std::ifstream f(path);
        if (!f) throw std::runtime_error("Cannot open " + path);
        return fromCsv(f);
    }

    int size() const { return static_cast<int>(names_.size()); }
    const std::vector<Edge>& edges() const { return edges_; }
    const std::string& name(int id) const { return names_.at(id); }
    const std::vector<std::pair<int, long long>>& neighbours(int u) const { return adj_.at(u); }

    int id(const std::string& name) const {
        auto it = index_.find(name);
        if (it == index_.end()) throw std::out_of_range("Unknown place: " + name);
        return it->second;
    }
    bool has(const std::string& name) const { return index_.count(name) > 0; }

private:
    std::vector<std::string> names_;
    std::map<std::string, int> index_;
    std::vector<std::vector<std::pair<int, long long>>> adj_;
    std::vector<Edge> edges_;
};

// ------------------------------------------------------------------ Dijkstra

struct DijkstraResult {
    std::vector<long long> dist;  // dist[v] = shortest distance in metres from source (INF if unreachable)
    std::vector<int> prev;        // prev[v] = previous place on the shortest path (-1 if none)
};

// One step of the algorithm, recorded for the dry run trace.
struct TraceStep {
    int settled;                                    // place removed from the queue and finalised
    long long settledDist;
    std::vector<std::tuple<int, long long, long long>> relaxed;  // (place, old dist, new dist)
};

// Dijkstra with a binary heap (std::priority_queue): O((V + E) log V).
inline DijkstraResult dijkstra(const Graph& g, int source, std::vector<TraceStep>* trace = nullptr) {
    int n = g.size();
    if (source < 0 || source >= n) throw std::out_of_range("Source index out of range");
    DijkstraResult r{std::vector<long long>(n, INF), std::vector<int>(n, -1)};
    std::vector<bool> done(n, false);
    using Item = std::pair<long long, int>;  // (distance, place)
    std::priority_queue<Item, std::vector<Item>, std::greater<Item>> pq;

    r.dist[source] = 0;
    pq.push({0, source});
    while (!pq.empty()) {
        auto [d, u] = pq.top();
        pq.pop();
        if (done[u]) continue;  // stale queue entry
        done[u] = true;
        TraceStep step{u, d, {}};
        for (auto [v, w] : g.neighbours(u)) {
            if (done[v]) continue;
            if (d + w < r.dist[v]) {
                step.relaxed.emplace_back(v, r.dist[v], d + w);
                r.dist[v] = d + w;
                r.prev[v] = u;
                pq.push({r.dist[v], v});
            }
        }
        if (trace) trace->push_back(step);
    }
    return r;
}

// Rebuild the path source -> target from prev[]. Empty if unreachable.
inline std::vector<int> buildPath(const DijkstraResult& r, int target) {
    std::vector<int> path;
    if (r.dist.at(target) >= INF) return path;
    for (int v = target; v != -1; v = r.prev[v]) path.push_back(v);
    std::reverse(path.begin(), path.end());
```

```
    return path;
}

// ------------------------------------------------------------- MST

struct MstResult {
    std::vector<Edge> edges;
    long long total = 0;
    bool spanning = false;  // false if the graph is disconnected (then it is a forest)
};

// Disjoint Set Union with path compression and union by rank.
class DSU {
public:
    explicit DSU(int n) : parent_(n), rank_(n, 0) {
        for (int i = 0; i < n; ++i) parent_[i] = i;
    }
    int find(int x) { return parent_[x] == x ? x : parent_[x] = find(parent_[x]); }
    bool unite(int a, int b) {
        a = find(a);
        b = find(b);
        if (a == b) return false;
        if (rank_[a] < rank_[b]) std::swap(a, b);
        parent_[b] = a;
        if (rank_[a] == rank_[b]) ++rank_[a];
        return true;
    }

private:
    std::vector<int> parent_, rank_;
};

// Kruskal: sort edges by weight, add each edge that joins two different components. O(E log E).
inline MstResult kruskal(const Graph& g) {
    MstResult r;
    std::vector<Edge> es = g.edges();
    std::stable_sort(es.begin(), es.end(), [](const Edge& a, const Edge& b) { return a.w < b.w; });
    DSU dsu(g.size());
    for (const Edge& e : es) {
        if (dsu.unite(e.u, e.v)) {
            r.edges.push_back(e);
            r.total += e.w;
        }
    }
    r.spanning = g.size() > 0 && static_cast<int>(r.edges.size()) == g.size() - 1;
    return r;
}

// Prim: grow one tree from `start`, always taking the cheapest edge leaving it. O(E log V).
inline MstResult prim(const Graph& g, int start = 0) {
    MstResult r;
    int n = g.size();
    if (n == 0) return r;
    std::vector<bool> inTree(n, false);
    using Item = std::tuple<long long, int, int>;  // (weight, from, to)
    std::priority_queue<Item, std::vector<Item>, std::greater<Item>> pq;
    pq.push({0, -1, start});
    while (!pq.empty()) {
        auto [w, from, to] = pq.top();
        pq.pop();
        if (inTree[to]) continue;
        inTree[to] = true;
        if (from != -1) {
            r.edges.push_back({from, to, w});
            r.total += w;
        }
        for (auto [v, wv] : g.neighbours(to))
            if (!inTree[v]) pq.push({wv, to, v});
    }
    r.spanning = static_cast<int>(r.edges.size()) == n - 1;
    return r;
}

}  // namespace mw
```

Appendix B: src/main.cpp

```
// main.cpp - Command line program for the Blantyre city road network
// (Limbe to Blantyre place). Distances are stored in metres and shown in km.
//
// Usage:
//   roads [--data FILE] path   <From> <To>    shortest route between two places
//   roads [--data FILE] trace  <From> <To>    same, plus a step-by-step dry run of Dijkstra
//   roads [--data FILE] all    <From>         shortest distance to every place
//   roads [--data FILE] mst    [prim|kruskal] minimum spanning tree
//   roads [--data FILE] places                list the places
//   roads [--data FILE]                      interactive menu
#include <iomanip>
#include <iostream>
#include <sstream>

#include "graph.hpp"

using namespace mw;

// Metres to km, two decimals only when needed: 1000 -> "1.0", 450 -> "0.45".
static std::string km(long long m) {
    if (m >= INF) return "inf";
    std::ostringstream o;
    o << std::fixed << std::setprecision(2) << m / 1000.0;
    std::string s = o.str();
    if (s.back() == '0') s.pop_back();
    return s;
}

static void printPath(const Graph& g, const std::string& from, const std::string& to, bool withTrace) {
    int s = g.id(from), t = g.id(to);
    std::vector<TraceStep> trace;
    DijkstraResult r = dijkstra(g, s, withTrace ? &trace : nullptr);

    if (withTrace) {
        std::cout << "Dry run of Dijkstra from " << from << "\n";
        int step = 1;
        for (const TraceStep& st : trace) {
            std::cout << "Step " << std::setw(2) << step++ << ": settle " << g.name(st.settled)
                      << " (" << km(st.settledDist) << " km)";
            if (st.relaxed.empty()) std::cout << ", no updates";
            for (auto [v, oldD, newD] : st.relaxed)
                std::cout << "\n          update " << g.name(v) << ": " << km(oldD) << " -> " << km(newD);
            std::cout << "\n";
            if (st.settled == t) {
                std::cout << "Target " << to << " settled, so its distance is final.\n";
                break;
            }
        }
        std::cout << "\n";
    }

    std::vector<int> path = buildPath(r, t);
    if (path.empty()) {
        std::cout << "No route from " << from << " to " << to << "\n";
        return;
    }
    std::cout << "Shortest route " << from << " -> " << to << ": " << km(r.dist[t]) << " km\n";
    for (size_t i = 0; i < path.size(); ++i) {
        std::cout << (i ? " -> " : "  ") << g.name(path[i]);
    }
    std::cout << "\n";
}

static void printAll(const Graph& g, const std::string& from) {
    DijkstraResult r = dijkstra(g, g.id(from));
    std::cout << "Shortest distances from " << from << ":\n";
    for (int v = 0; v < g.size(); ++v) {
        std::cout << "  " << std::left << std::setw(29) << g.name(v) << std::right << std::setw(6)
                  << km(r.dist[v]) << " km   via ";
        std::vector<int> p = buildPath(r, v);
        for (size_t i = 0; i < p.size(); ++i) std::cout << (i ? " > " : "") << g.name(p[i]);
        std::cout << "\n";
    }
}

static void printMst(const Graph& g, const std::string& algo) {
    MstResult m;
```

```
    if (algo == "prim") m = prim(g);
    else if (algo == "kruskal") m = kruskal(g);
    else throw std::invalid_argument("MST algorithm must be prim or kruskal");
    std::cout << "Minimum spanning tree (" << algo << "), in the order edges were added:\n";
    for (const Edge& e : m.edges)
        std::cout << "  " << std::left << std::setw(29) << g.name(e.u) << " - " << std::setw(29) << g.name(e.v)
                  << std::right << std::setw(5) << km(e.w) << " km\n";
    std::cout << "Edges: " << m.edges.size() << ", total length: " << km(m.total) << " km"
              << (m.spanning ? "" : "  (graph is disconnected: this is a spanning forest)") << "\n";
}

static void printPlaces(const Graph& g) {
    std::cout << g.size() << " places, " << g.edges().size() << " roads\n";
    for (int v = 0; v < g.size(); ++v) std::cout << "  " << g.name(v) << "\n";
}

static void interactive(const Graph& g) {
    while (true) {
        std::cout << "\n1) Shortest route  2) Route with dry run  3) Distances from a place\n"
                     "4) MST (Prim)  5) MST (Kruskal)  6) List places  0) Quit\nChoice: ";
        std::string c;
        if (!std::getline(std::cin, c) || c == "0") return;
        try {
            std::string a, b;
            if (c == "1" || c == "2" || c == "3") {
                std::cout << "From place: ";
                std::getline(std::cin, a);
            }
            if (c == "1" || c == "2") {
                std::cout << "To place: ";
                std::getline(std::cin, b);
            }
            if (c == "1") printPath(g, a, b, false);
            else if (c == "2") printPath(g, a, b, true);
            else if (c == "3") printAll(g, a);
            else if (c == "4") printMst(g, "prim");
            else if (c == "5") printMst(g, "kruskal");
            else if (c == "6") printPlaces(g);
            else std::cout << "Unknown choice\n";
        } catch (const std::exception& e) {
            std::cout << "Error: " << e.what() << "\n";
        }
    }
}

int main(int argc, char** argv) {
    std::vector<std::string> args(argv + 1, argv + argc);
    std::string data = "data/blantyre_roads.csv";
    if (args.size() >= 2 && args[0] == "--data") {
        data = args[1];
        args.erase(args.begin(), args.begin() + 2);
    }
    try {
        Graph g = Graph::fromFile(data);
        if (args.empty()) interactive(g);
        else if (args[0] == "path" && args.size() == 3) printPath(g, args[1], args[2], false);
        else if (args[0] == "trace" && args.size() == 3) printPath(g, args[1], args[2], true);
        else if (args[0] == "all" && args.size() == 2) printAll(g, args[1]);
        else if (args[0] == "mst") printMst(g, args.size() > 1 ? args[1] : "kruskal");
        else if (args[0] == "places") printPlaces(g);
        else {
            std::cerr << "Usage: roads [--data FILE] path|trace <From> <To> | all <From> | mst [prim|kruskal] |
places\n";
            return 2;
        }
    } catch (const std::exception& e) {
        std::cerr << "Error: " << e.what() << "\n";
        return 1;
    }
    return 0;
}
```

Appendix C: MST program output

```
$ ./roads mst kruskal
Minimum spanning tree (kruskal), in the order edges were added:
  Mandala                       - Blantyre Market              0.21 km
  Kamba Pork Market             - Stereo Night Club             0.3 km
  CI Primary School             - Blantyre Market               0.3 km
  Limbe Health Centre           - Liquid Lounge Roundabout     0.35 km
  Limbe Market                  - Limbe Health Centre          0.45 km
  Liquid Lounge Roundabout      - MASM                         0.55 km
  Red Cross Malawi              - Mandala                      0.55 km
  Stereo Night Club             - Rosewood Mall                 0.7 km
  Automotive Products Limited   - St Columba                   0.85 km
  Liquid Lounge Roundabout      - Puma Energy Kanjedza          1.0 km
  MASM                          - Yanakis Roundabout            1.0 km
  Reunion Insurance Roundabout  - Independence Arch             1.0 km
  Yanakis Roundabout            - Reunion Insurance Roundabout  1.1 km
  Kamba Pork Market             - Red Cross Malawi              1.2 km
  Blantyre Clock Tower          - PA Nandos                     1.2 km
  PA Nandos                     - Blantyre Market               1.2 km
  Castel                        - HHI                           1.4 km
  Automotive Products Limited   - Chichiri Shopping Mall        1.5 km
  Puma Energy Kanjedza          - Automotive Products Limited   1.6 km
  Chichiri Shopping Mall        - Castel                        1.9 km
  Chichiri Shopping Mall        - Red Cross Malawi              2.3 km
Edges: 21, total length: 20.66 km
```